PCA Three Ways: SVD, Eigendecomposition and a Linear Autoencoder

Course: Computational Linear Algebra (MA30241) · Course Project Report

Group: Ponna Sai Nithish (24MA10052) · Sarvesh Mahajan (24MA10060) · Sidhant Mishra (24MA10064) · Shaurya Pratap (24MA10061)

Date: 10 November 2026

Abstract

All three methods find the same principal subspace, but at very different cost. We computed principal component analysis (PCA) three ways: the singular value decomposition (SVD) of the data, the eigendecomposition of the covariance matrix, and a linear autoencoder trained by gradient descent. We compared complexity, time, memory, reconstruction error, subspace agreement and scalability.

On handwritten digits (N = 1797, d = 61, k = 21), all three reached the Eckart–Young optimal relative error of 0.3111. SVD and eigendecomposition agreed to 3×10^{-15}, and the autoencoder's subspace was within 0.98° of PCA. Eigendecomposition was fastest (0.85 ms, against 5.5 ms for SVD and 526 ms for the autoencoder). SVD was the most numerically robust. The autoencoder used the least memory and its cost grew linearly with d, but it was only approximate and needed tuning.

1. Introduction and problem statement

PCA finds the k orthonormal directions along which centred data varies most. Projecting onto them gives the best k-dimensional linear summary of the data. Two classical linear-algebra routes compute these directions exactly: the SVD of the data matrix, and the eigendecomposition of its covariance matrix.

A third route comes from neural networks: an autoencoder trained to reproduce its input through a bottleneck of k hidden units. Bourlard and Kamp (1988) proved that, with linear output units, the optimal autoencoder performs exactly the PCA projection.

Goal. Compare the three methods on:

1. computational complexity
2. execution time
3. memory consumption
4. reconstruction accuracy
5. how closely the trained autoencoder approximates the PCA subspace

6. scalability as the number of features d grows.

2. Mathematical background

PCA is a best low-rank approximation problem, solved by the truncated SVD. Let $X \in \mathbb{R}^{(N \times d)}$ hold N samples as rows, centred so that each column has mean zero.

SVD. Every X can be written as a sum of rank-1 matrices, where r = rank(X) and U, V have orthonormal columns:

$X = U\Sigma V^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \ldots + \sigma_r u_r v_r^T$, with $\sigma_1 \geqslant \sigma_2 \geqslant \ldots \geqslant \sigma_r > 0$

Variance and the Rayleigh quotient. The variance of the data along a unit vector v is $v^T C v$, with $C = X^T X / (N - 1)$. It is largest along the first right singular vector:

max over $v \neq 0$ of $(v^T X^T X v) / (v^T v) = \sigma_1^2$, attained at $v = v_1$

Repeating the maximisation orthogonal to the directions already found gives v_2, v_3, … So the principal directions are the right singular vectors, and the variance along v_i is $\lambda_i = \sigma_i^2 / (N - 1)$.

Eckart–Young. The truncated SVD is the best rank-k approximation:

$X_k = \sigma_1 u_1 v_1^T + \ldots + \sigma_k u_k v_k^T$

min over $\text{rank}(B) \leqslant k$ of $\|X - B\| = \|X - X_k\| = \sqrt{(\sigma^2_{k+1} + \ldots + \sigma^2_r)}$ (Frobenius norm)

The PCA reconstruction $X V_k V_k^T$ equals X_k, so it has the smallest error any k-component method can reach. This gives an exact benchmark for all three methods.

Choosing k. We take the smallest k that keeps 90% of the energy:

$(\sigma_1^2 + \ldots + \sigma_k^2) / (\sigma_1^2 + \ldots + \sigma_r^2) \geqslant 0.90$

For the digits data this gives k = 21, which keeps 90.3% of the variance.

3. The three methods

Method 1: SVD of the data. Compute the thin SVD $X = U\Sigma V^T$ with `np.linalg.svd(X, full_matrices=False)`. This calls LAPACK's gesdd: Householder bidiagonalisation, then divide-and-conquer on the bidiagonal matrix. The method gives:

- directions: the first k columns of V
- variances: $\sigma_i^2 / (N - 1)$
- scores: $Z = XV_k = U_k \Sigma_k$

Method 2: eigendecomposition of the covariance. Form $C = X^TX / (N - 1)$ and diagonalise it with the symmetric solver `np.linalg.eigh`: $C = Q\Lambda Q^T$. Since $X^TX = V\Sigma^2V^T$, $Q = V$ up to the sign of each column, and $\lambda_i = \sigma_i^2 / (N - 1)$.

The catch is conditioning. Forming X^TX squares the condition number, $\kappa(X^TX) = \kappa(X)^2$, so directions with σ_i/σ_1 below about $\sqrt{\varepsilon_mach} \approx 10^{-8}$ are lost.

Method 3: linear autoencoder. A two-layer network with no activation function. The encoder gives $z = W_1x$, with $W_1 \in \mathbb{R}^(k\times d)$, and the decoder gives $\hat{x} = W_2z$, with $W_2 \in \mathbb{R}^(d\times k)$. It is trained to minimise the total squared reconstruction error:

$E(W_1, W_2) = \Sigma_n \|W_2W_1x_n - x_n\|^2$, summed over $n = 1, \ldots, N$

W_2W_1 has rank at most k, so minimising E is a best rank-k approximation problem. Bourlard and Kamp (1988) show that the optimum is the following, for any invertible $k \times k$ matrix T:

$W_2 = V_kT^{-1}$, $W_1 = TV_k^T$, so $W_2W_1 = V_kV_k^T$

The consequences:

- The network learns the PCA subspace, but its basis is neither orthonormal nor ordered by variance (Bishop & Bishop, 2024, §19.1.1).
- Its codes are $z = T \cdot$ (PCA scores): mixed and rescaled PCA coordinates.
- The loss has no spurious local minima (Baldi & Hornik, 1989), so gradient descent can reach this optimum.
- Sigmoid hidden units do not improve on it (Bourlard & Kamp, §5).
- The optimal output bias only reproduces the data mean, so we centre the data and use no biases.

Training. Back-propagation with Adam, mini-batches of 32 and 100 epochs. The learning rate decays geometrically from 0.05 to 0.0003. Weights start from $N(0, 1/d)$ for the encoder and $N(0, 1/k)$ for the decoder.

4. Complexity and memory

The two classical methods share the same order of cost; they differ in constants and memory. The autoencoder grows only linearly in d, but that cost is repeated for every epoch. Notation: $N \geqslant d$ samples, k components, E epochs, batch size B; memory is counted beyond the input X.

Method	Time	Memory	Main steps
SVD	$O(Nd^2)$	$O(Nd + d^2)$	Householder bidiagonalisation ($\approx 4Nd^2 - 4d^3/3$ flops), SVD of the bidiagonal matrix, then forming the thin $N \times d$ factor U
Eigendecomposition	$O(Nd^2 + d^3)$	$O(d^2)$	Forming C ($\approx Nd^2$ multiply-adds, one matrix product), then a symmetric $d \times d$ eigensolve ($O(d^3)$)
Linear autoencoder	$O(E \cdot Ndk)$	$O(dk + Bd)$	Each epoch: five matrix products of size $N \times d \times k$ (two forward, three backward)

Eigendecomposition has the same order as SVD but is faster in practice, for two reasons:

- Forming X^TX is a single, highly optimised matrix product.
- It never builds the $N \times d$ factor U, which is also why it needs much less memory.

5. Experimental setup

We used one real and one synthetic dataset, ran all three methods on the same centred data, and recorded five measurements.

Dataset 1: digits (real). scikit-learn's digits dataset: 1797 grey-scale images of handwritten digits 0–9, each 8×8 pixels, with intensities 0–16 scaled to 0–1.

- Three pixels are blank in every image. They carry no information and make X rank-deficient, which leaves the autoencoder's encoder weights undetermined in those directions, so we drop them.
- This gives $N = 1797$ and $d = 61$, with condition number ≈ 660 and $k = 21$.

Dataset 2: synthetic (scalability). $N = 2000$ points in $d \in \{50, 100, 200, 400, 800, 1600\}$ dimensions, generated as $X = ZA^T + 0.1 \cdot \text{noise}$.

- A has 10 orthonormal columns, and the 10 hidden factors have standard deviations from 3 down to 1.
- The true subspace is therefore 10-dimensional, so $k = 10$.

Measurements.

Measure	How it was measured
Time	Fastest of up to 5 runs (autoencoder: 1–2 runs), with time.perf_counter

Measure	How it was measured
Peak memory	tracemalloc during the call: NumPy arrays plus LAPACK workspace; the input itself is not counted
Reconstruction error	$\|X - \hat{X}\|_F / \|X\|_F$, compared with the Eckart–Young optimum
Subspace match	Largest principal angle between the method's subspace and the SVD subspace (scipy subspace_angles); for the autoencoder, the span of the decoder W_2
Scalability	All of the above, repeated for each d

Environment. Python with NumPy 2.5 (OpenBLAS) on a 2-core Intel Xeon at 2.1 GHz. All code and outputs are in the accompanying notebook, PCA_SVD_Eig_Autoencoder.ipynb.

6. Results

6.1 Digits ($N = 1797, d = 61, k = 21$)

All three methods reached the optimal error, but their running times differed by two to three orders of magnitude.

Method	Time	Peak memory	Relative error	Largest angle to SVD subspace
SVD	5.5 ms	2.82 MB	0.3111	0° (reference)
Eigendecomposition	0.85 ms	0.16 MB	0.3111	$\approx 10^{-12}$°
Linear autoencoder	526 ms	0.18 MB	0.3111	0.98°

- The Eckart–Young optimum is 0.311129; the autoencoder is only 0.0045% above it.
- The SVD and eigendecomposition variances agree to 3.4×10^{-15} (relative).
- Eigendecomposition was about 6.5× faster than SVD; the autoencoder was about 95× slower than SVD.
- Absolute timings vary between runs and machines; the ratios are stable.

6.2 Does the autoencoder find the PCA subspace?

Yes, but in a different basis.

- **Subspace:** the decoder subspace is within 0.98° of the PCA subspace (mean principal angle 0.23°). The encoder weights are within 1.54°.
- **Basis:** it is not orthonormal. The diagonal of $W_2^T W_2$ ranges from 5.47 to 9.24 (it would be 1 if orthonormal), and off-diagonal entries reach 1.37.
- Each learned vector is therefore a mixture of principal components, as the arbitrary T in the Bourlard–Kamp solution predicts.

6.3 Scalability with d (synthetic, N = 2000, k = 10)

SVD and eigendecomposition grow roughly like d^2; the autoencoder grows roughly linearly in d.

d	SVD time (s)	Eig time (s)	AE time (s)	SVD memory (MB)	Eig memory (MB)	AE memory (MB)	AE angle (°)
50	0.0039	0.0007	0.304	2.5	0.11	0.10	0.10
100	0.0103	0.0014	0.395	5.3	0.41	0.19	0.12
200	0.0259	0.0047	0.520	11.5	1.6	0.35	0.14
400	0.0949	0.0204	0.825	26.9	6.4	0.69	0.22
800	0.4230	0.1233	1.582	69.2	25.7	1.35	0.27
1600	2.2832	0.7921	3.182	179.4	102.6	2.53	1.81

- **Growth rates.** Fitting cost $\propto d^p$ over d = 400–1600 gives:
  - time: p = 2.3 (SVD), 2.6 (eigendecomposition), 1.0 (autoencoder)
  - memory: p = 1.4, 2.0 and 0.9 respectively.
- **Why the time exponents exceed 2:** as d approaches N, the d^3 terms of SVD and eigendecomposition begin to count.
- **Speed gap:** the autoencoder is 78× slower than SVD at d = 50, but only 1.4× slower at d = 1600.
- **Memory:** at d = 1600 the autoencoder needs 40–70× less memory than the classical methods.
- **Autoencoder accuracy:** with a fixed 100 epochs, its subspace angle stays ⩽ 0.27° up to d = 800 but rises to 1.81° at d = 1600.

6.4 Numerical precision (Läuchli matrix)

SVD keeps singular values that eigendecomposition destroys. We take the matrix below with $\delta = 10^{-8}$; its exact singular values are $\sqrt{(3 + \delta^2)}$, δ and δ.

$X = [1\,1;\ \delta\,0;\ 0\,\delta;\ 0\,0\ \delta]$, $X^TX = [1+\delta^2\ 1;\ 1\ 1+\delta^2\ 1;\ 1\ 1\ 1+\delta^2]$ (rows separated by semicolons)

Singular value	Exact	SVD	√(eig of X^TX)
σ_1	1.732051	1.732051	1.732051
σ_2	1×10^{-8}	1×10^{-8}	0 (100% error)
σ_3	1×10^{-8}	1×10^{-8}	0 (100% error)

Why eigendecomposition fails here:

- $\delta^2 = 10^{-16}$ is below half of machine epsilon (2.2×10^{-16}), so $1 + \delta^2$ rounds to 1.
- The computed X^TX is then the all-ones matrix, which has rank 1.
- $\kappa(X) \approx 1.7 \times 10^8$, so $\kappa(X^TX) \approx 3 \times 10^{16}$, which exceeds $1/\varepsilon_mach$.

On the digits ($\kappa \approx 660$) this effect is negligible, which is why the two classical methods agreed there.

7. Discussion

No method wins on every measure. SVD is the safe default, eigendecomposition is fastest when $N \gg d$, and the autoencoder pays in speed and exactness for small memory.

	SVD	Eigendecomposition	Linear autoencoder
Accuracy	Exact; most stable (never forms X^TX)	Exact for well-conditioned data; loses directions with $\sigma_i/\sigma_1 \lesssim 10^{-8}$	Approximate (≈ 1° here); depends on epochs and learning rate
Output	Orthonormal, ordered directions, plus variances and scores	Same, up to sign	Subspace only: basis mixed by T, no variances
Speed	Middle	Fastest in every test	Slowest; the gap shrinks as d grows
Memory	Largest (N × d factor U)	$O(d^2)$	Smallest, $O(dk + Bd)$
Best for	Accuracy, ill-conditioned data	$N \gg d$	Data too large for memory, streaming data, non-linear extensions

scikit-learn's PCA follows the same logic (version 1.9 source). It switches to covariance eigendecomposition when $d \leqslant 1000$ and $N \geqslant 10d$, and otherwise uses full or randomized SVD.

Limitations.

- Timings come from one 2-core machine. Absolute values vary; the ratios are more stable.
- The autoencoder used a fixed budget of 100 epochs at every d, and its accuracy fell at $d = 1600$.
- Its encoder weights converge more slowly than its decoder. The encoder gradient is multiplied by XX^T, so it moves very slowly along low-variance directions.
- The encoder part of the Bourlard–Kamp result needs full-rank data, which is why we dropped the blank pixels.
- tracemalloc counts memory allocated through NumPy, not the Python interpreter's own memory.

8. Conclusion

All three methods compute the same principal subspace.

- **Agreement:** SVD and eigendecomposition agree to machine precision. The linear autoencoder converges to the same subspace, as Bourlard and Kamp proved, and gets within 0.0045% of the Eckart–Young error.
- **SVD** is the reference: exact, ordered, orthonormal and numerically safest, at the price of memory.
- **Eigendecomposition** is fastest when $N \gg d$, but it squares the condition number.
- **The linear autoencoder** is the slowest and only approximate for linear PCA. Yet its tiny memory, linear cost in d and mini-batch training are what make its non-linear (deep) extensions practical.

References

1. H. Bourlard and Y. Kamp, "Auto-association by multilayer perceptrons and singular value decomposition", *Biological Cybernetics* 59 (1988) 291–294. Electronic reprint: IDIAP Research Report IDIAP-RR 00-16, July 2000.
2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, Chapter 19 (§19.1, Deterministic autoencoders).
3. P. Baldi and K. Hornik, "Neural networks and principal component analysis: learning from examples without local minima", *Neural Networks* 2 (1989) 53–58.

4. L. N. Trefethen and D. Bau, *Numerical Linear Algebra*, SIAM, 1997.
5. MA30241 Computational Linear Algebra, class notes: SVD, low-rank approximation and eigenvalue computation.
6. Code and outputs: PCA_SVD_Eig_Autoencoder.ipynb (NumPy, SciPy, scikit-learn).